

IOTA

Decentralized blockchain for the Cloud-Edge-IoT continuum

Salvador Cuñat Negueroles

Rafael Vaño Garcia

salcuane@upv.es

ravagar2@upv.es

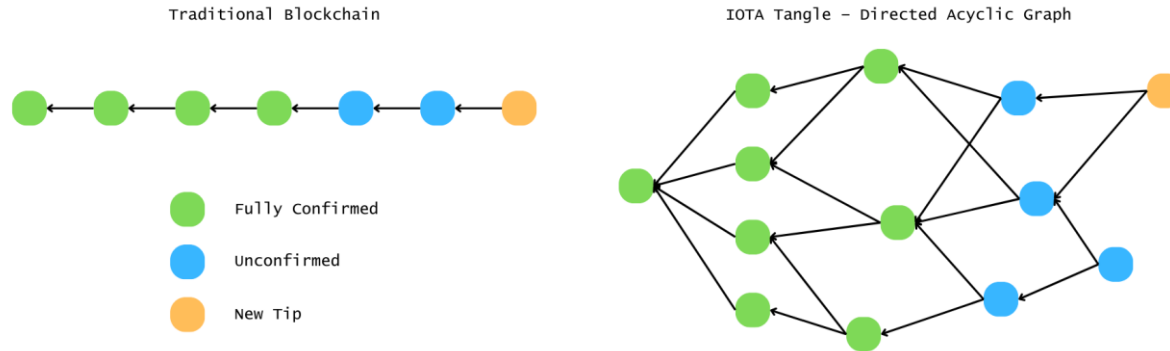
Universitat Politècnica de València (UPV)



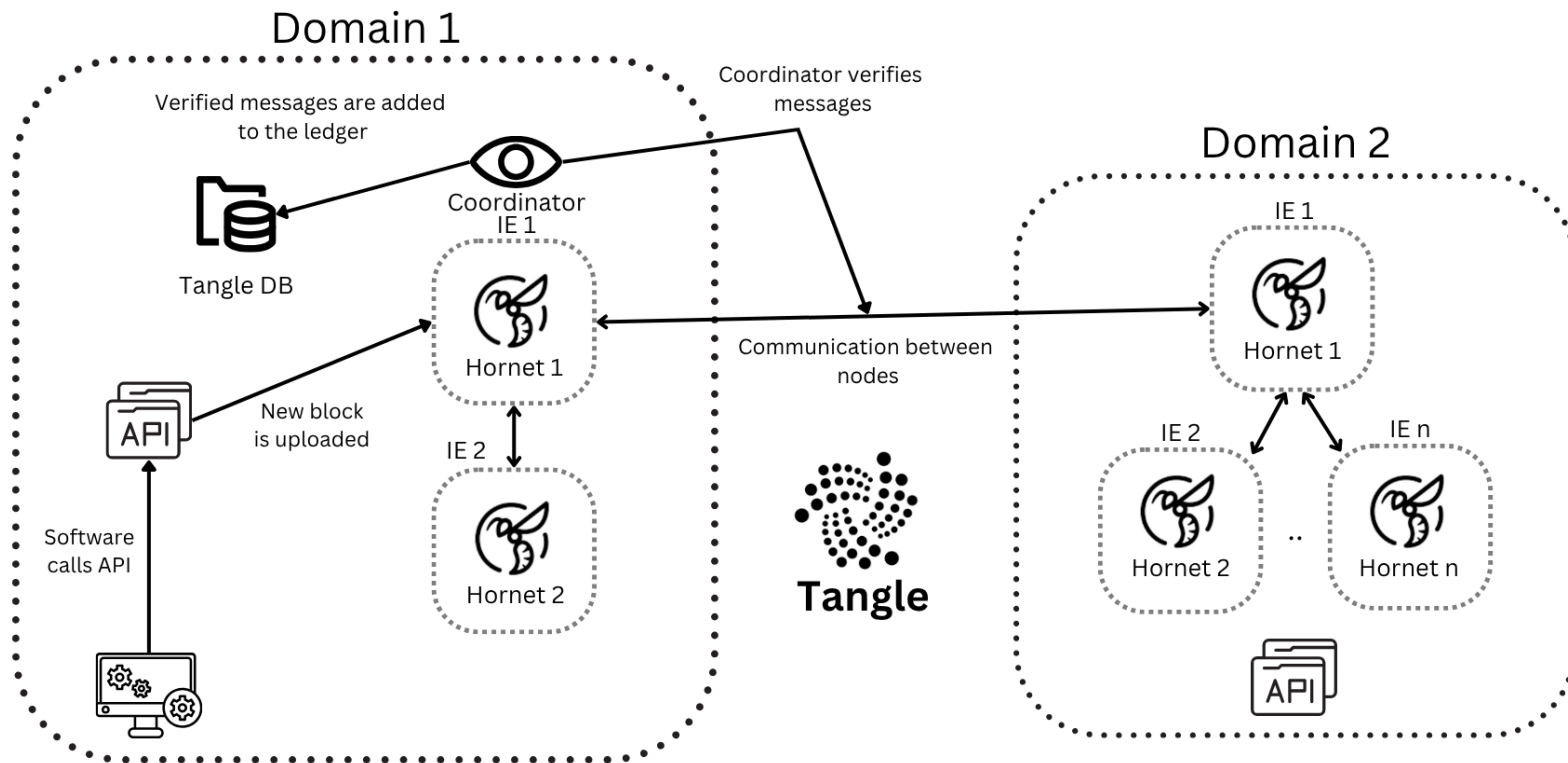
Funded by
the European Union

What is IOTA

- IOTA is a **Distributed Ledger Technology** which allows for a **trustworthy** fee-less data transactions between different nodes in the Tangle.
 - The Tangle is a data structure replicated across a network of nodes.
 - It has all the information necessary to track messages and ensure traceability of payloads.
 - Enables parallel validation of transactions.
 - Designed around the Directed Acyclic Graph (DAG) theory.

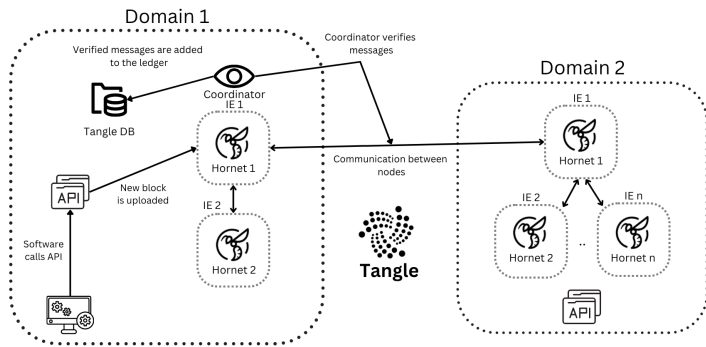


How does IOTA work



How does IOTA work

- **Hornet** nodes can be installed in different machines in the network.
 - From here messages – A.K.A. “blocks” – can be uploaded to the network.
- The **Coordinator** is a “plugin” attached to a node that sends **signed blocks** called **milestones**.
 - These are used to confirm blocks and reach consensus.
 - A new block needs to reference previously accepted blocks.
- Other plugins used so far for testing are:
 - A very basic Python API to more easily upload messages.
 - A dashboard for visualization of messages and peering of nodes
- Your work will be on **improving** those **functionalities**.
- The versions of IOTA used here are the “Stardust” versions, latest official documentation will most likely not be compatible.



Challenge introduction and guidelines

- **After successfully installing a basic IOTA Tangle** (with at least one hornet – the coordinator) at your computer, and checking that (1) the UI is visible at <http://localhost:31011> and (2) the hornet API is reachable at <http://localhost:14625>
- **Basic goal** (mandatory) – **IOTA Advanced Explorer**: Build an observability/traceability system that contains human-digestible information. To achieve this goal, you must build (at least) two applications:
 1. One that is able to **send a message to the Tangle** (insert a message via the hornet API, you can use the [Eclipse aeriOS IOTA Messages API](#)).
 2. One that stores the messages inserted to the Tangle in a **parallel database** (recommended: relational database) that contains enhanced information about such message (e.g. timestamps, event tag...), and that evaluates whether the message received is legit and which content it has. This requires:
 - a) **Modify** the **Messages API** to send messages also to your application
 - b) **Retrieve** (including search parameters: per ID, date, tag...) automatically (via REST API) the content of a message inserted in the Tangle.
 - c) **Check** that each message (with a given *blockId*) is indeed **valid** (solid) inside the Tangle (using the *GET block metadata* endpoint of the *hornet API*).
 - d) **Check** that the **content** of one message (e.g., that arrives to my application) is indeed the correct one (using the *GET block endpoint* of the *hornet API*).

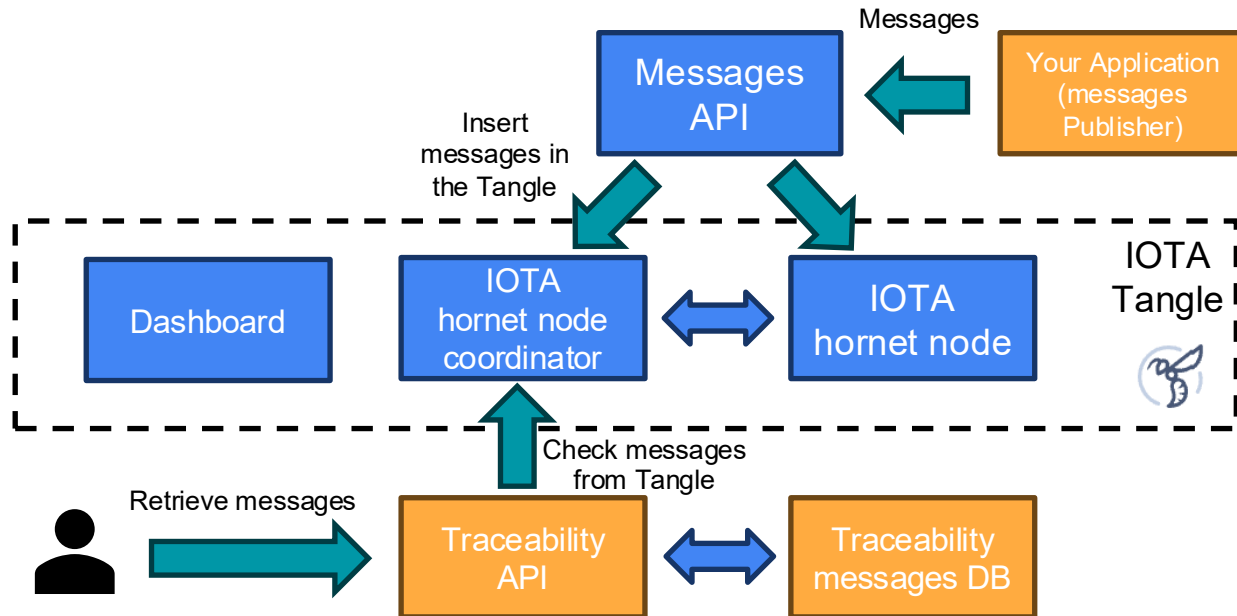
Some ideas for the challenge

- 
- **Idea #1 - MQTT:** Instead of retrieving the data from the Tangle via the API, deploy a MQTT broker that would: (1) receive a publication from IOTA API every time a new message is inserted – requires to modify the API, and, (2) your own application that subscribes to the proper topic to receive a notification in real time as messages are posted in the broker.
 - **Idea #2 – Explorer UI:** Departing from the basic goal, develop a parallel simple UI dashboard that will make visible the enriched content of the solid messages.
 - **Idea #3 – IoT traceability:** Think and develop an approach for being able to track related messages. In an IoT application, many sensors/users can insert messages in the Tangle. You must propose methodological and technical approach for being able to track messages that correspond to the same flow of events / user / IoT sensor...
 - **Idea #4 – Verifiable receipts of critical events:** Design and Build an **Incident Explorer** that reconstructs key trust-related events stored in the IOTA Tangle. The system groups correlated events into a chronological timeline and verifies each one against its corresponding BlockID. It can also generate **real-time notifications or alerts** when critical events occur, helping operators quickly understand what happened and react accordingly.

Feel free to explore further options to deliver a more sophisticated outcome!

The jury will bear in consideration the level of complexity of the final product pitched on Thursday.

Architecture diagram of the challenge



Useful endpoints and installation requirements

- IOTA hornet API:
 - GET /api/core/v2/<block-id>
 - GET /api/core/v2/<block-id>/metadata
- Messages API:
 - POST /upload?node=<your-local-IP>
- IOTA Tangle Dashboard:
 - <http://localhost:31011/>

```
{
  "tag": "self.reorquestation",
  "message": {
    "format": "I just need a valid JSON",
    "be_creative": true
  }
}
```

- **Docker** and Docker Compose installed
- **Linux is required.** If you are using Windows, create and use a Linux distribution using [Windows Subsystem for Linux \(WSL\)](#)

Useful Links

<https://github.com/eclipse-aerios/iota-tangle>

<https://github.com/eclipse-aerios/iota-messages-api>

<https://legacy.wiki.iota.org/iota-sdk/welcome/>

<https://github.com/iotaedger/horner>

https://github.com/iotaedger/horner/tree/develop/private_tangle

<https://github.com/iotaedger/iota-sdk>

<https://github.com/iotaedger/inx-dashboard>



We are here to help you!

Good coding 😊